MoneyPoly: Code Quality Analysis & White-Box Testing Report

Roll No: 2024101063

1 Code Quality Analysis (Pylint)

1.1 Requirement coverage

This section addresses the requirement:
"Use pylint to analyze the codebase. Make sure to iteratively fix the code as per the warnings and suggestions. Document all the changes in the report. Save each iteration as a commit in the format `Iteration #: <What You Changed>`".

1.2 Project layout notes

During development, the project layout was reorganized. In particular, commit `10e4ebc` moved the runnable package under `whitebox/code/moneypoly` (and the entrypoint under `whitebox/code/main.py`).

1.3 How pylint was run

Because the importable package lives under `whitebox/code`, pylint was executed from that directory:

```
cd whitebox/code
python -m pylint moneypoly --score=y
```

1.4 Final pylint status

On the final state of the `main` branch, pylint reports:

- **Score:** 10.00/10

1.5 Iteration-by-iteration change log

Each pylint-driven cleanup pass was saved as an `Iteration #` commit. The table below documents **all iteration commits**, their hashes, and the **files touched** in each iteration. (The intent and file lists are taken directly from the commit history.)

Iter.	Hash	Commit subject	Files changed / what was updated
1	377b37e	Board Module fixed	Updated board implementation (`board.py`) to satisfy pylint and improve module hygiene.
2	1ae7ab5	Config Module fixed	Updated configuration module (`config.py`) for lint compliance and clarity.

Iter.	Hash	Commit subject	Files changed / what was updated
3	ed46260	Property module fixed	Updated `board.py` and `property.py` to address lint findings across property/group interactions.
4	522ec59	UI module fixed	Updated UI helpers (`ui.py`) to satisfy pylint and standardize formatting/inputs.
5	a570c26	Game module simple warnings fixed	Updated game loop (`game.py`); also introduced `whitebox/bugs_fixed.md` as a running log.
6	fd81804	Cards module fixed and gitignore added	Updated cards/deck logic (`cards.py`); added `.gitignore`; removed committed cache artifacts.
7	2778194	Dice module fixed + removed import error	Updated multiple modules (bank/board/dice/game/player) to resolve lint/import issues exposed by dice changes.
8	590ffab	Bank module fixed	Updated `bank.py` and documentation log for consistent validation and clean API behavior.
9	d9ff76b	Player module basic warnings fixed	Updated `player.py` and documentation log for cleaner state handling and lint compliance.
10	e951079	Game module unused imports errors fixed	Updated `board.py` and `game.py` to remove unused imports and remaining lint-only issues.
11	10e4ebc	Final cleanup + refactor; moved code under whitebox	Refactored and reorganized: moved package to `whitebox/code/moneypoly` and entrypoint to `whitebox/code/main.py`.

Note: The goal of the iteration commits is not feature work; each iteration is a focused lint-driven cleanup of the indicated module(s). The report records the exact commit boundaries so the grader can verify the iterative process.

2 White-Box Test Cases

2.1 Requirement coverage

This section addresses the requirement:
"Design white-box test cases based on the structure of the code. Your tests must cover: (i) All branches, (ii) Key variable states that could affect program behavior, (iii) Relevant edge cases. In the report, explain in simple words why each test case is needed, along with all the errors or logical issues found by the test."

2.2 Where the tests live (as per directory requirement)

Per the required directory structure, the white-box test suite is located at:

- `whitebox/tests`

2.3 How tests were run

Because the importable package lives under `whitebox/code`, the test runner is executed from `whitebox/` and `whitebox/tests/conftest.py` adds `whitebox/code` to the import path:

```
cd whitebox
python -m pytest -q tests
```

2.4 Test-case catalog (why each test is needed)

This subsection explains **each test case** in simple words: what branch/state/edge-case it targets and why that matters.

Bank — `whitebox/tests/test_bank.py`

`test_bank_initialization`
: Checks initial state (reserves/loan counters). This is the baseline for every later Bank branch. (Issue found: none)

`test_bank_collect`
: Covers the positive-amount collect branch; ensures balance increases correctly. (Issue found: none)

`test_bank_pay_out`
: Covers the normal payout branch; ensures value returned and funds decrease. (Issue found: none)

`test_bank_pay_out_negative_or_zero`
: Edge-case: payout of 0/negative must be a no-op to prevent corrupting balances. (Issue found: none)

`test_bank_pay_out_insufficient_funds`
: Branch: bank must refuse to overpay; validates exception path. (Issue found: none)

`test_bank_give_loan`
: State path: loan changes both player balance and bank accounting; also validates printed output. (Issue found: none)

`test_bank_give_loan_negative_or_zero`
: Edge-case: invalid loan amounts must not create loan records. (Issue found: none)

`test_bank_summary_and_repr`
: Ensures reporting helpers remain stable for debugging and UI logs. (Issue found: none)

Board — `whitebox/tests/test_board.py`

`test_board_initialization`
: Confirms the board loads properties and groups; sanity-checks tile typing at position 0. (Issue found: none)

`test_get_tile_type`
: Covers the decision paths for many tiles and the out-of-range fallback. (Issue found: none)

`test_get_property_at`
Covers returning a Property vs `None` for non-property/out-of-bounds. (Issue found: none)

`test_is_purchasable`
Covers branches: non-property, owned, and mortgaged states. (Issue found: none)

`test_is_special_tile`
Covers special-tile decision logic and blank fallback. (Issue found: none)

`test_ownership_queries`
Ensures board ownership aggregations are correct (used by game/UI). (Issue found: none)

`test_board_repr`
Ensures repr remains useful/stable for debugging. (Issue found: none)

Cards — `whitebox/tests/test_cards.py`

`test_card_deck_initialization`
Validates initial deck state and cards-remaining computation. (Issue found: none)

`test_card_deck_draw_and_peek`
Covers index changes and the exhaustion boundary; ensures reshuffle happens before the next cycle. (Issue found: Error 9)

`test_card_deck_empty`
Edge-case: empty deck must return `None` without crashing. (Issue found: none)

`test_card_deck_reshuffle`
Covers reshuffle branch; ensures index reset and size preserved. (Issue found: none)

`test_card_deck_repr`
Ensures repr includes size and next index for debugging. (Issue found: none)

Dice — `whitebox/tests/test_dice.py`

`test_dice_initialization`
Baseline dice state; sanity-checks helpers derived from default values. (Issue found: none)

`test_dice_roll_bounds`
Ensures dice outputs remain in 1–6 and totals 2–12 for all paths. (Issue found: Error 3)

`test_dice_doubles_streak`
Covers doubles streak increment/reset branch with deterministic random. (Issue found: none)

`test_dice_reset`
Ensures reset clears all dice state to avoid cross-turn leakage. (Issue found: none)

`test_dice_describe`
Covers output formatting for doubles/non-doubles. (Issue found: none)

`test_dice_repr`
Ensures repr remains stable for debugging. (Issue found: none)

Player — `whitebox/tests/test_player.py`

`test_player_initialization`
Baseline player state (money, position, jail flags) used by many game branches. (Issue found: none)

`test_player_initialization_custom_balance`
Covers constructor branch for custom balances. (Issue found: none)

`test_add_money`
Normal add-money branch. (Issue found: none)

`test_add_money_negative_raises_valueerror`
Edge-case: reject negative additions. (Issue found: none)

`test_deduct_money`
Normal deduct-money branch. (Issue found: none)

`test_deduct_money_negative_raises_valueerror`
Edge-case: reject negative deductions. (Issue found: none)

`test_is_bankrupt`
Covers bankruptcy decision boundary at 0 and negative values. (Issue found: none)

`test_net_worth`
Baseline net worth for winner logic. (Issue found: none)

`test_move_no_wrap`
Branch: no GO crossing means no salary. (Issue found: none)

`test_move_with_wrap`
Branch: wrap-around must award GO salary and print message. (Issue found: Error 1)

`test_move_past_go`
Branch: crossing GO by more than board size must still award salary. (Issue found: Error 1)

`test_go_to_jail`
State transition: set jail flags and move to jail position. (Issue found: none)

`test_add_and_remove_property`
Covers dedup/remove-missing branches for property list management. (Issue found: none)

`test_status_line`
Covers jailed vs non-jailed formatting branch. (Issue found: none)

`test_repr`
Ensures repr stability. (Issue found: none)

`test_state_properties`
Covers getters/setters for key state variables. (Issue found: none)

Property — `whitebox/tests/test_property.py`

`test_property_initialization`
Baseline property state affecting rent/mortgage branches. (Issue found: none)

`test_property_mortgage_value`
Validates mortgage value calculation used in mortgage cashflow. (Issue found: none)

`test_property_mortgage`
Covers mortgage branch + repeated mortgage no-op edge-case. (Issue found: none)

`test_property_unmortgage`
Covers unmortgage branch + repeated unmortgage no-op edge-case. (Issue found: none)

`test_property_is_available`
Covers availability branches: unowned/owned/mortgaged. (Issue found: none)

`test_property_repr`
Covers owned vs unowned repr branch. (Issue found: none)

`test_property_get_rent_basic`
Covers base rent and mortgaged rent=0 branch. (Issue found: none)

`test_property_get_rent_full_group`
Covers full-group doubling branch and regression to base rent when group breaks. (Issue found: Error 2)

`test_property_group_basic`
Covers group bookkeeping and duplicate-add edge-case. (Issue found: none)

`test_property_group_all_owned_by`
Covers ownership decision branches for partial ownership. (Issue found: none)

`test_property_group_owner_counts`
Covers aggregation of owner counts (reporting/analysis). (Issue found: none)

UI — `whitebox/tests/test_ui.py`

`test_print_banner`
Smoke-test for output formatting. (Issue found: none)

`test_print_player_card`
Covers printing when jailed and holding jail cards. (Issue found: none)

`test_print_player_card_with_properties`
Covers printing when properties exist and are mortgaged. (Issue found: none)

`test_print_standings`
Covers jailed annotation branch in standings output. (Issue found: none)

`test_print_board_ownership`
Covers mortgaged marker branch in ownership table. (Issue found: none)

`test_format_currency`
Edge-case: large numbers formatting. (Issue found: none)

`test_safe_int_input`
Edge-case: invalid/empty input falls back to default instead of crashing. (Issue found: none)

`test_confirm`
Covers parsing branches for yes/no input. (Issue found: none)

Game — `whitebox/tests/test_game.py`

`test_game_initialization`
Baseline game state (players, counters, running flag). (Issue found: none)

`test_advance_turn`
Covers round-robin wrap-around and turn-number update. (Issue found: none)

`test_find_winner`
Covers winner computation branch (max net worth). (Issue found: Error 4)

`test_play_turn_jail`
Covers jailed-player branch when player declines to pay/use card. (Issue found: none)

`test_play_turn_doubles_to_jail`
Covers three-doubles-to-jail rule branch. (Issue found: none)

`test_play_turn_invokes_interactive_menu_in_tty`
Ensures pre-roll menu is reachable during normal play (interactive only). (Issue found: Error 8)

`test_buy_property_skip`
Covers unowned property decision branch: skip. (Issue found: none)

`test_buy_property_buy`
Covers unowned property decision branch: buy. (Issue found: none)

`test_buy_property_auction`
Covers unowned property decision branch: auction. (Issue found: none)

`test_auction_logic`
Covers auction branch where a bidder wins. (Issue found: none)

`test_auction_logic_all_pass`
Edge-case: all players pass; property remains unowned. (Issue found: none)

`test_pay_rent`
Covers rent payment branch and transfer of money to owner. (Issue found: Error 6)

`test_mortgage_and_unmortgage_commands`
Covers mortgage/unmortgage state transitions. (Issue found: none)

`test_trade_logic`
Covers trade cash + ownership transfer. (Issue found: Error 7)

`test_bankruptcy_skips`
Regression: ensure player elimination does not skip the next player. (Issue found: Error 5)

`test_apply_card_actions`
Covers card action dispatch branches (collect/pay/jail/jail-free/move/birthday). (Issue found: none)

3 Errors / Logical Issues Found and Fixed

The table below lists each discovered issue (via the tests above), the fixing commit, and the short explanation of what was wrong.

Err.	Hash	Primary test trigger	What was wrong (simple words) and what was changed
1	c374dc3	`test_move_with_wrap/test_move_past_go`	Passing/landing on GO did not reliably award salary; fixed movement logic to add GO salary when wrapping.
2	d9ef01c	`test_property_get_rent_full_group`	Rent doubling logic for complete ownership sets was incorrect; fixed rent calculation to double only when group ownership condition holds.
3	9e52b9f	`test_dice_roll_bounds`	Dice roll range missed the value 6; fixed randomness bounds so dice results are 1–6.
4	ab3ac14	`test_find_winner`	Winner selection used the wrong comparator (min instead of max); fixed to return player with maximum net worth.
5	69ff35c	`test_bankruptcy_skips`	Removing a bankrupt player could mis-adjust `current_index` and skip the next player; fixed index update on elimination.
6	a7b0105	`test_pay_rent`	Rent was not being transferred correctly to the property owner; fixed rent transfer to credit the owner.
7	3c28703	`test_trade_logic`	Trade did not correctly transfer cash to the seller; fixed trade cashflow and ownership updates.
8	6d1706b	`test_play_turn_invokes_interactive_menu_in_tty`	Mortgage/trade/menu logic existed but was unreachable from normal play; integrated menu into the turn flow for interactive runs.
9	a09756e	`test_card_deck_draw_and_peek`	Card decks cycled without reshuffling, making draws predictable after one full cycle; reshuffle now occurs automatically on exhaustion.